ЛАБОРАТОРНА РОБОТА №12
Функціональне програмування:
Вбудовані функції. Невеликі функції та lambda-вирази.
Функції модулів itertools, functools, operator.

Мета: дослідити основні техніки функціонального програмування засобами мови програмування Python.

Загальні вимоги до виконання лабораторної роботи:
- Написати консольний додаток, що реалізує завдання згідно умови ***мовою Python.***
- Вхідні дані подаються користувачем на консоль.
- Результати відображаються користувачу у консольному режимі.

Індивідуальні завдання

Завдання 1.1. ***Сортування за ключем***

Вам надано список словників, які представляють людей. Кожен словник містить ключі "name" (ім'я) та "age" (вік). Реалізуйте функцію sort_by_age(), яка повертає новий список, відсортований за значенням "age" за зростанням.

Сигнатура функції:

def sort_by_age(people: list[dict]) -> list[dict]:

Template:

```
def sort_by_age(people: list[dict]) -> list[dict]:
    return []

print(
    sort_by_age(
        [
            {"name": "Alice", "age": 30},
            {"name": "Bob", "age": 25},
            {"name": "Eve", "age": 35},
        ]
    )
)
```

Підказки щодо реалізації:
1) Використайте вбудовану функцію sorted(iterable, key).
2) Передайте аргумент key у вигляді lambda, який витягує значення "age" із кожного словника.
3) Не змінюйте список на місці — повертайте новий список.

Приклади:

```
sort_by_age([
...     {"name": "Alice", "age": 30},
...     {"name": "Bob", "age": 25},
...     {"name": "Eve", "age": 35}
])
```

Результат: [{'name': 'Bob', 'age': 25}, {'name': 'Alice', 'age': 30}, {'name': 'Eve', 'age': 35}]

```
sort_by_age([
...     {"name": "Zara", "age": 18},
...     {"name": "Liam", "age": 22},
...     {"name": "Noah", "age": 20}
])
```

Результат: [{'name': 'Zara', 'age': 18}, {'name': 'Noah', 'age': 20}, {'name': 'Liam', 'age': 22}]

<u>Завдання 1.2.</u> ***Фільтр довгих слів***

Напишіть функцію filter_long_words(words: list[str]) -> list[str], яка повертає новий список, що містить лише ті слова з вхідного списку, довжина яких більша за 3 символи. Ви обов'язково повинні використати вбудовану функцію filter().

Template:

```
def filter_long_words(words: list[str]) -> list[str]:
    # Write your solution using filter() here.
    return []

print(filter_long_words(["a", "the", "code", "Python", "is",
"fun"]))
```

Підказки до реалізації:

1) Скористайтесь lambda-функцією всередині filter(function, iterable), щоб перевіряти довжину кожного слова.
2) Результат filter() потрібно перетворити на список за допомогою list().
3) Слова з рівно 3 літерами не повинні включатися до результату.
4) Порожні рядки ігноруються.

Приклади:

- filter_long_words(["a", "the", "code", "Python", "is", "fun"]) #['code', 'Python']
- filter_long_words(["cat", "dog", "fish", "go", "egg"]) #['fish']
- filter_long_words(["", "aa", "bbb", "cccc", "ddddd"]) #['cccc', 'ddddd']
- filter_long_words([]) #[]

<u>Завдання 1.3.</u> ***Кожне слово з великої літери***

Реалізуйте функцію capitalize_words(), яка приймає ітерабельну послідовність рядків (наприклад, список, кортеж або будь-який інший об'єкт, що підтримує ітерацію), і повертає нову ітерабельну послідовність, де кожен рядок починається з великої літери, а решта літер – малі. Функція повинна використовувати вбудовану функцію map(function, iterable).

Сигнатура функції:

def capitalize_words(words: Iterable[str]) -> Iterable[str]:

Template:

```
from typing import Iterable
def capitalize_words(words: Iterable[str]) -> Iterable[str]:
    # Реалізуйте використовуючи map() і str.capitalize
    return words

print(list(capitalize_words(["python", "java", "c++"])))
```

Принципи конструювання ПЗ

Підказки до реалізації:

1) Скористайтесь функцією str.capitalize().
2) Не використовуйте генератори чи спискові включення.
3) Поверніть результат одразу з map(...), без перетворення на список або інший тип.

Приклади:

- list(capitalize_words(["python", "java", "c++"])) #['Python', 'Java', 'C++']
- tuple(capitalize_words(("hello", "world"))) #('Hello', 'World')
- list(capitalize_words([""])) #['']
- list(capitalize_words([])) #[]

<u>Завдання 1.4.</u> ***Перевірка пароля на правила***

Напишіть функцію validate_password(password: str) -> bool, яка перевіряє, чи введений пароль відповідає всім заданим вимогам безпеки.

Вимоги до паролю (правила):

- містить хоча б одну велику літеру (A-Z);
- містить хоча б одну цифру (0-9);
- має довжину не менше 8 символів;
- містить хоча б один спеціальний символ з набору !@#$%^&*().
- не містить пробілів.

Функція повинна повертати True, якщо пароль проходить усі перевірки, інакше – False.

Template:

```
def validate_password(password: str) -> bool:
    # Write your solution here.
    return False

print(validate_password("StrongPass1!"))
```

Підказки до реалізації:

1) Для кожного правила створіть окрему функцію, що приймає password: str і повертає bool. Функції повинні мати назви:
   - has_uppercase
   - has_digit
   - is_long_enough
   - has_special_char
   - no_spaces
2) Створіть список правил типу list[Callable[[str], bool]].
3) Для перевірки всіх правил використайте вбудовану функцію all(...).
4) Для правил "містить хоча б одну" можна використовувати вбудовану функцію any(...).
5) Можна використовувати методи str.isupper(), str.isdigit().

Приклади:

- validate_password("StrongPass1!") #True
- validate_password("weakpass") #False
- validate_password("Short7!") #False
- validate_password("With Space1!") #False
- validate_password("NoSpecialChar1") #False